REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Josefina Pino Monsalve

7 de abril de 2026

00:43

1. Entorno experimental

Para analizar la complejidad de los algoritmos es necesario describir las especificaciones tanto del hardware como del entorno de software, de manera que los experimentos puedan ser reproducidos en condiciones similares. A continuación, se detalla la infraestructura utilizada:

Hardware:

- Memoria RAM: 16 GB DDR4 @ 2666 MHz
- Procesador: Intel(R) Core(TM) i3-10100F CPU @ 3.60 GHz
- Almacenamiento: Disco duro de 932 GB y SSD de 223 GB

Software:

- Intérprete de comandos: GNU Bash 5.1.16 (x86_64-pc-linux-gnu)
- Sistema Operativo: Ubuntu 22.04.3 LTS
- Entorno de ejecución: Windows Subsystem for Linux (WSL) sobre Visual Studio Code
- Compilador: g++ (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
- Compilación:
    - `-std=c++17 -O2`

***Para este informe no se utilizaron los arreglos de tamaño 10^7 ni tampoco las matrices de tamaño 1024, ya que el tiempo de ejecución era muy elevado.**

2. Resultados y Análisis

En esta sección se presentan los resultados obtenidos a partir de la ejecución de los programas desarrollados. Estos resultados se representan en gráficas que permiten visualizar y analizar el comportamiento de los algoritmos implementados. Antes de analizar los algoritmos de ordenamiento, se muestra un resumen de las complejidades teoricas del tiempo de ejecucion de los distintos algoritmos.

Complejidad teórica:

- Merge Sort: O(n log n) peor caso y promedio.
- Quick Sort: O(n^2) peor caso, O(n log n) promedio.
- Sort (std::sort): O(n log n) promedio y peor caso.

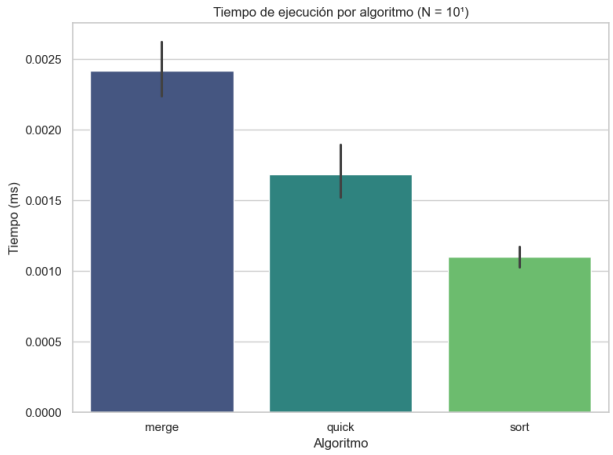

Figura 1: N = 10

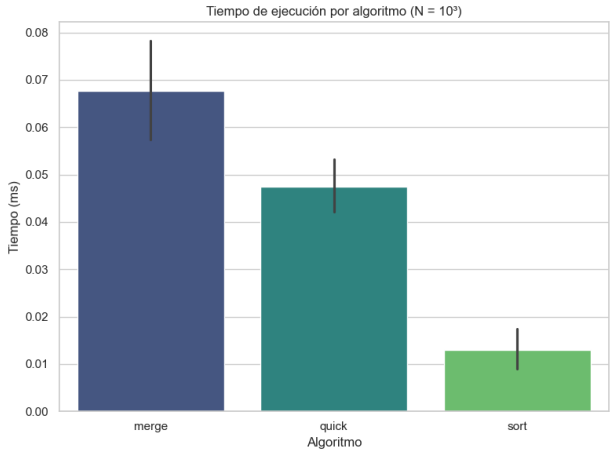

Figura 2: N = 1000

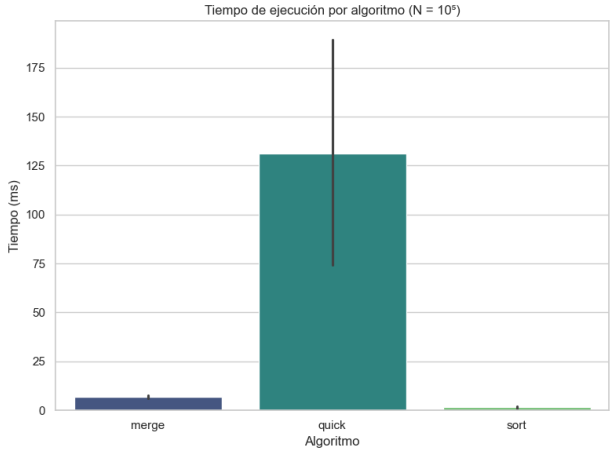

Figura 3: N = 100000

Análisis de tiempos por tamaño de arreglo (N) : Fig. 1, 2 y 3.

- **N = 10:** Merge Sort presenta el mayor tiempo de ejecución, mientras que Quick Sort y Sort muestran los menores tiempos. Debido al tamaño reducido del arreglo, todos los algoritmos tienen tiempos muy bajos (máximo ≈ 0,0025 ms), pero comparándolos se observan diferencias significativas.

- **N = 1.000:** Al analizar con un aumento de N, podemos notar que sigue la misma tendencia que con $N = 10$, pero con tiempos mas elevados (alrededor de ≈ 0,07 ms para Merge Sort). Además se observa que Sort destaca con un tiempo pequeño (≈ 0,01 ms).

- **N = 100.000:** Quick Sort es el que tiene el tiempo de ejecución más elevado (≈ 125 ms, con alta variabilidad), mientras que Merge Sort y Sort tienen tiempos menores a 5 ms y cercanos a 0. Esto se explica por la dependencia de Quick Sort respecto a la elección del pivote, lo que puede degradar su rendimiento en ciertos casos.

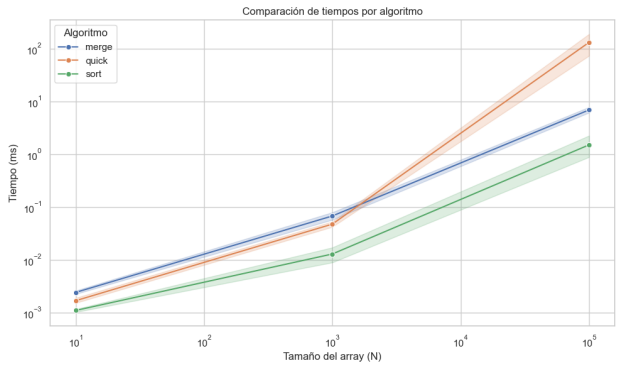

Figura 4: Tiempo por algoritmo

Análisis de crecimiento de tiempo vs tamaño (N) : Fig. 4.

Se observa claramente cómo los tiempos de ejecución aumentan con el tamaño del arreglo. Merge Sort y Sort presentan una tendencia de crecimiento similar (**O(n log n)**), siendo Sort el más eficiente de los dos, lo cual refleja optimizaciones adicionales. El Quick Sort tiene un aumento en el tiempo de ejecución en arreglos mayores de 10^5 principalmente por el pivote.

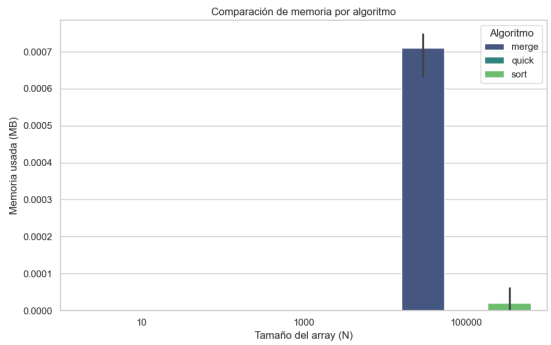

Figura 5: Memoria por algoritmo

Análisis de memoria vs tamaño (N) : Fig. 5.

- Arreglos de tamaño 10 y 1.000: Todos los algoritmos ocupan aproximadamente la misma memoria, debido a la el tamaño pequeño de los arreglos de cada implementación.
- Arreglos de tamaño 100.000: Merge Sort utiliza una gran cantidad de memoria en comparación con los otros algoritmos. Sort ocupa memoria cercana al 0, mientras que Quick Sort no ocupa memoria extra.

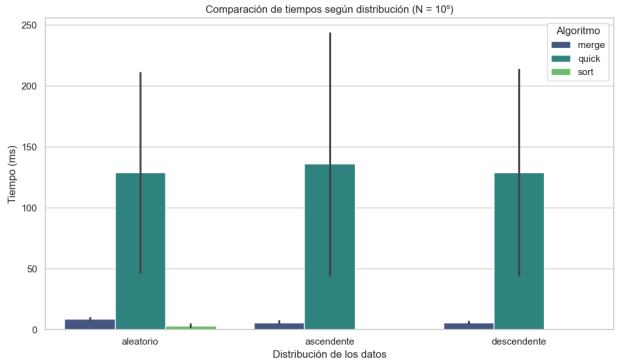

Figura 6: Memoria por algoritmo

Análisis de tiempo vs distribución (Aleatoria, ascendente y descendente) : Fig. 6. Al analizar como afecta la distribución se puede apreciar que para los algoritmos Sort y Merge sort no hay mucha diferencia en los tiempos. También se mantiene esto en Quick Sort pero con una varianza en todas las distribuciones, por lo que es relativo el tiempo que toma en ordenar el arreglo.

Antes de analizar los algoritmos de multiplicacion, se muestra un resumen de las complejidades teoricas del tiempo de ejecucion de los distintos algoritmos.

Marco teórico:

- **Naive:** complejidad temporal de $O(n^3)$, derivada de los tres bucles anidados que recorren filas y columnas.
- **Strassen:** reduce la complejidad a aproximadamente $O(n^{2,81})$ mediante la descomposición recursiva y uso de 7 multiplicaciones de submatrices en lugar de 8. Aunque es teóricamente más eficiente, en dimensiones pequeñas la sobrecarga de la recursión provoca que sea más lento que el método clásico.

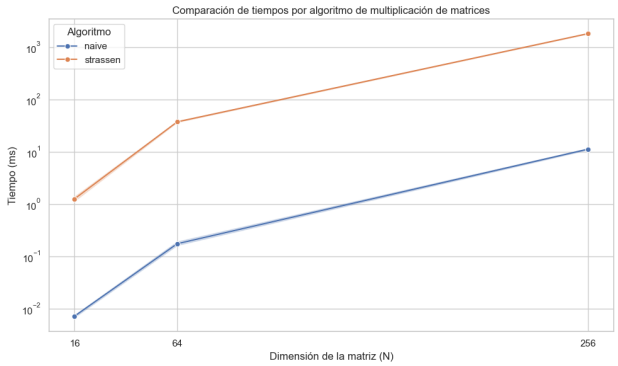

Figura 7: Tiempo por algoritmo

Análisis de tiempo (T) vs tamaño de matriz (N): Fig. 6.

En la comparación del tiempo de ejecución según la dimensión de la matriz, ambos algoritmos muestran un crecimiento similar, lo cual es consistente con que ambos tienen complejidad superior a lineal.

No obstante, se observa que el algoritmo **Naive** comienza con valores de tiempo más bajos que Strassen para dimensiones pequeñas. Esto ocurre porque el método de Strassen introduce sobrecarga por la recursión y las operaciones adicionales necesarias para dividir y combinar submatrices.

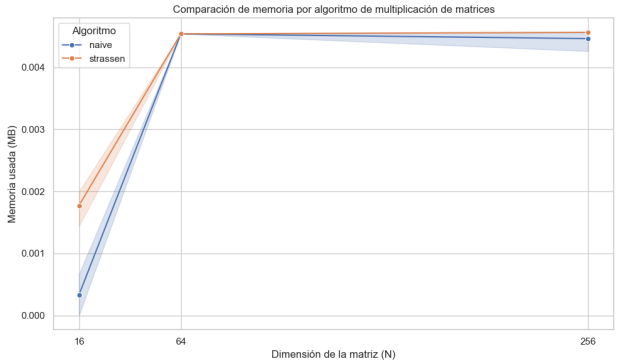

Figura 8: Memoria por algoritmo

Análisis de uso de memoria vs tamaño de la matriz (N)

En la comparación de memoria utilizada, se observa un aumento pronunciado al pasar de matrices de dimensión 16 a 64. Sin embargo, después de ese punto, el crecimiento se estabiliza, mostrando un patrón más controlado. Además, se aprecia que el algoritmo **Naive** requiere menos memoria que Strassen en todas las dimensiones evaluadas. Esto se debe a que Strassen utiliza estructuras adicionales para almacenar submatrices y realizar llamadas recursivas.

- **Naive:** el uso de memoria se limita a las matrices de entrada y salida, manteniéndose en $O(n^2)$.
- **Strassen:** requiere espacio adicional para submatrices temporales en cada nivel de recursión, lo que incrementa el uso de memoria hasta $O(n^2 \log n)$ en algunas implementaciones prácticas.

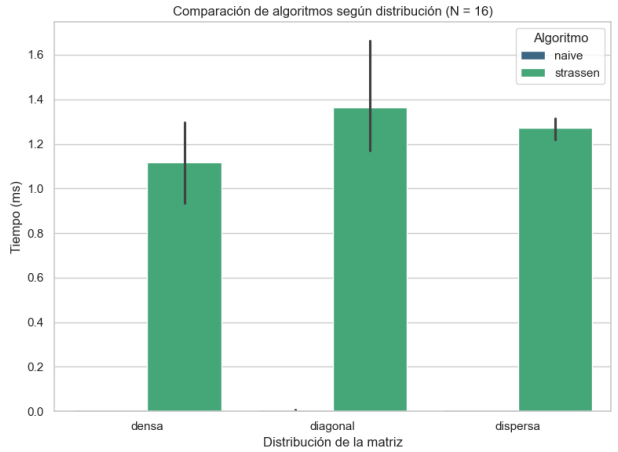

Figura 9: Tiempo según distribución

Para el algoritmo **Naive** no existe una diferencia en la distribución pero para **Strassen** existe una pequeña diferencia de tiempo, siendo la diagonal la más compleja de procesar.

Los resultados muestran que, aunque Strassen es teóricamente más eficiente en tiempo asintótico, para dimensiones pequeñas el algoritmo Naive resulta más rápido y más ligero en memoria.

3. Conclusiones

Los resultados obtenidos en este estudio muestran que la teoría de la complejidad algorítmica se refleja de manera consistente en la práctica, aunque con algunas diferencias. En el caso de los algoritmos de ordenamiento, se confirmó que algoritmos $O(n \log n)$ como Merge Sort, Sort, y Quick Sort escalan de forma más eficiente, solo que este último puede variar más según la distrubució y tamaño. Esto se debe por la posición del pivote que genera aleatoriedad. Entre ellos, Sort se posiciona como la mejor alternativa, gracias a las optimizaciones de su implementación.

En la multiplicación de matrices, Strassen demuestra su superioridad teórica sobre el método Naive al reducir la complejidad a $O(n^{2,81})$. No obstante, en tamaños pequeños su sobrecarga recursiva hace que Naive sea más competitivo, lo que muestra que la eficiencia práctica depende no solo de la complejidad asintótica, sino también de la estructura del algoritmo y del contexto de ejecución.

A un nivel más general, este trabajo evidencia la relevancia del análisis experimental como complemento indispensable de la teoría en el aprendizaje de este curso. Las medidas de tiempo y memoria confirman que la elección del algoritmo adecuado no depende únicamente de su clasificación teórica, sino también del tamaño de la entrada, la naturaleza de los datos y la infraestructura disponible. De este modo, los hallazgos contribuyen a un mejor entendimiento de cómo teoría y práctica se entrelazan en la computación, y resaltan la necesidad de evaluar algoritmos en escenarios reales antes de aplicarlos.